

PPL Week-6

Rachit Takate – 230962294 – B54

I. Write a program in CUDA which performs convolution operation on 1D input array N of size width using a mask array M of size mask width to produce the resultant 1D array P of size width.

```
#include <stdio.h>
#include <stdlib.h>
#include <cuda.h>

__global__ void conv1D(float *arr, float *ker, float *res, unsigned int mLen,
unsigned int len) {
    int i = blockDim.x * blockIdx.x + threadIdx.x;
    int start = i - mLen / 2;

    float val = 0;
    for(int j = 0; j < mLen; j++) {
        if(start + j >= 0 && start + j < len)
            val += arr[start + j] * ker[j];
    }

    res[i] = val;
}

void printArrf(float *arr, unsigned int len) {
    for(unsigned int i = 0; i < len; i++)
        printf("%.0f ", arr[i]);
    printf("\n");
}

int main() {
    float *arr, *ker, *res;
    float *dArr, *dKer, *dRes;
    int n, m;

    printf("Array size: ");
    scanf("%d", &n);
    printf("Mask size: ");
    scanf("%d", &m);
    size_t nSize = sizeof(float) * n, mSize = sizeof(float) * m;

    res = (float *) malloc(nSize);
    arr = (float *) malloc(nSize);
    ker = (float *) malloc(mSize);
    for(int i = 0; i < n; i++) {
        arr[i] = rand() % 10;
        if(i < m)
            ker[i] = rand() % 10;
    }

    printf("Array:\n");
    printArrf(arr, n);
    printf("Kernel:\n");
    printArrf(ker, m);

    cudaMalloc((void **) &dRes, nSize);
    cudaMalloc((void **) &dArr, nSize);
    cudaMalloc((void **) &dKer, mSize);
```

```

    cudaMemcpy(dArr, arr, nSize, cudaMemcpyHostToDevice);
    cudaMemcpy(dKer, ker, mSize, cudaMemcpyHostToDevice);

    conv1D<<<1, n>>>(dArr, dKer, dRes, m, n);
    cudaMemcpy(res, dRes, nSize, cudaMemcpyDeviceToHost);

    printf("Result:\n");
    printArrf(res, n);

    cudaFree(dArr);
    cudaFree(dKer);
    cudaFree(dRes);

    printf("Rachit 54\n");
    return 0;
}

```

Output

```

(base) mca@computinglab26-26:~/230962294/PP Lab/Week6$ nvcc 01.cu
(base) mca@computinglab26-26:~/230962294/PP Lab/Week6$ ./a.out
Array size: 51
Mask size: 5
Array:
3 7 3 6 9 2 7 0 9 3 6 0 6 2 6 1 8 7 9 2 0 2 3 7 5 9 2 2 8 9 7 3 6 1 2 9 3 1 9 4 7 8 4 5 0 3 6 1 0 6 3
Kernel:
6 5 5 2 1
Result:
32 62 89 107 104 105 108 68 99 72 105 62 76 53 86 70 104 101 132 99 71 35 44 81 98 118 97 99 87 114 140 117 91 66 72 68 83 96 83 93 129 112 112 96 61 58 47 59 56 42 45
Rachit 54

```

II. Write a program in CUDA to perform selection sort in parallel.

```

#include <stdio.h>
#include <stdlib.h>
#include <cuda.h>

__global__ void selectSort(float *arr, float *res, unsigned int len) {
    int i = blockDim.x * blockIdx.x + threadIdx.x;

    if(i < len) {
        unsigned int rank = 0;
        for(unsigned int j = 0; j < len; j++)
            if(arr[i] > arr[j] || (arr[i] == arr[j] && j < i))
                rank++;

        res[rank] = arr[i];
    }
}

void printArrf(float *arr, unsigned int len) {
    for(unsigned int i = 0; i < len; i++)
        printf("%.0f ", arr[i]);
    printf("\n");
}

int main() {
    float *arr;
    float *dArr, *dRes;
    int n;

    printf("Array size: ");
    scanf("%d", &n);
    size_t len = sizeof(float) * n;

    arr = (float *) malloc(len);
    for(int i = 0; i < n; i++)
        arr[i] = rand() % 100;
}

```

```

printf("Unsorted:\n");
printArrf(arr, n);

cudaMalloc((void **) &dArr, len);
cudaMalloc((void **) &dRes, len);

cudaMemcpy(dArr, arr, len, cudaMemcpyHostToDevice);

selectSort<<<(len - 1) / 256 + 1, 256>>>(dArr, dRes, n);
cudaMemcpy(arr, dRes, len, cudaMemcpyDeviceToHost);

printf("Sorted:\n");
printArrf(arr, n);

cudaFree(dArr);
cudaFree(dRes);

printf("Rachit 54\n");
return 0;
}

```

Output

```

(base) mca@computinglab26-26:~/230962294/PP_Lab/Week6$ nvcc Q2.cu
(base) mca@computinglab26-26:~/230962294/PP_Lab/Week6$ ./a.out
Array size: 200
Unsorted:
83 86 77 15 93 35 86 92 49 21 62 27 90 59 63 26 40 26 72 36 11 68 67 29 82 30 62 23 67 35 29 2 22 58 69 67 93 56 11 42 29 73 21 19 84 37 98 24 15 70 13 26 91 80 56 73 62 70 96 81 5 25 84
27 36 5 46 29 13 57 24 95 82 45 14 67 34 64 43 50 87 8 76 78 88 84 3 51 54 99 32 60 76 68 39 12 26 86 94 39 95 70 34 78 67 1 97 2 17 92 52 56 1 80 86 41 65 89 44 19 40 29 31 17 97 71 81
75 9 27 67 56 97 53 86 65 6 83 19 24 28 71 32 29 3 19 70 68 8 15 40 49 96 23 18 45 46 51 21 55 79 88 64 28 41 50 93 0 34 64 24 14 87 56 43 91 27 65 59 36 32 51 37 28 75 7 74 21 58 95 29
37 35 93 18 28 43 11 28 29
Sorted:
0 1 1 2 2 3 3 5 5 6 7 8 8 9 11 11 12 13 13 14 14 15 15 15 17 17 18 18 19 19 19 19 21 21 21 21 22 23 23 24 24 24 24 25 26 26 26 26 27 27 27 27 28 28 28 28 28 29 29 29 29 29 29 30
31 32 32 32 34 34 34 35 35 35 36 36 36 37 37 37 39 39 40 40 40 41 41 42 43 43 43 44 45 45 46 46 49 49 50 50 51 51 51 52 53 54 55 56 56 56 56 57 58 58 59 59 60 62 62 62 63 64 64 65
65 65 67 67 67 67 67 68 68 68 69 70 70 70 71 71 72 73 73 74 75 75 76 76 77 78 78 79 80 80 81 81 82 82 83 83 84 84 84 86 86 86 86 86 87 87 88 88 89 90 91 91 92 92 93 93 93 93 94 95
95 95 96 96 97 97 97 98 99
Rachit 54

```

III. Write a program in CUDA to perform odd even transposition sort in parallel.

```

#include <stdio.h>
#include <stdlib.h>
#include <cuda.h>

__global__ void oddEvenSort(float *arr, unsigned int len, bool isOdd) {
    int i = blockDim.x * blockIdx.x + threadIdx.x;

    if(i < len && i % 2 == isOdd) {
        float temp;
        if(arr[i] > arr[i + 1]) {
            temp = arr[i];
            arr[i] = arr[i + 1];
            arr[i + 1] = temp;
        }
    }
}

void printArrf(float *arr, unsigned int len) {
    for(unsigned int i = 0; i < len; i++)
        printf("%.0f ", arr[i]);
    printf("\n");
}

int main() {
    float *arr;
    float *dArr;
    int n;

    printf("Array size: ");
    scanf("%d", &n);

```

```

size_t len = sizeof(float) * n;

arr = (float *) malloc(len);
for(int i = 0; i < n; i++)
    arr[i] = rand() % 100;

printf("Unsorted:\n");
printArrf(arr, n);

cudaMalloc((void **) &dArr, len);
cudaMemcpy(dArr, arr, len, cudaMemcpyHostToDevice);

for(int i = 0; i < n; i++) {
    oddEvenSort<<<(len - 1) / 256 + 1, 256>>>(dArr, n, i % 2);
    cudaDeviceSynchronize();
}
cudaMemcpy(arr, dArr, len, cudaMemcpyDeviceToHost);

printf("Sorted:\n");
printArrf(arr, n);

cudaFree(dArr);

printf("Rachit 54\n");
return 0;
}

```

Output

```

(base) mca@computinglab26-26:~/230962294/PP_Lab/Week6$ nvcc Q3.cu
(base) mca@computinglab26-26:~/230962294/PP_Lab/Week6$ ./a.out
Array size: 200
Unsorted:
83 86 77 15 93 35 86 92 49 21 62 27 90 59 63 26 40 26 72 36 11 68 67 29 82 30 62 23 67 35 29 2 22 58 69 67 93 56 11 42 29 73 21 19 84 37 98 24 15 70 13 26 91 80 56 73 62 70 96 81 5 25 84
27 36 5 46 29 13 57 24 95 82 45 14 67 34 64 43 50 87 8 76 78 88 84 3 51 54 99 32 60 76 68 39 12 26 86 94 39 95 70 34 78 67 1 97 2 17 92 52 56 1 80 86 41 65 89 44 19 49 29 31 17 97 71 81
75 9 27 67 56 97 53 86 65 6 83 19 24 28 71 32 29 3 19 70 68 8 15 40 49 96 23 18 45 46 51 21 55 79 88 64 28 41 50 93 0 34 64 24 14 87 56 43 91 27 65 59 36 32 51 37 28 75 7 74 21 58 95 29
37 35 93 18 28 43 11 28 29
Sorted:
0 0 1 1 2 2 3 3 5 5 6 7 8 8 9 11 11 11 12 13 13 14 14 15 15 15 17 17 18 18 19 19 19 21 21 21 21 22 23 23 24 24 24 24 25 26 26 26 26 27 27 27 27 28 28 28 28 28 29 29 29 29 29 29
30 31 32 32 32 34 34 34 35 35 35 36 36 36 37 37 37 39 39 40 40 40 41 41 42 43 43 43 44 45 45 46 46 49 49 50 50 51 51 51 52 53 54 55 56 56 56 56 57 58 58 59 59 60 62 62 62 63 64 64 64
65 65 65 67 67 67 67 67 68 68 68 69 70 70 70 70 71 71 72 73 73 74 75 75 76 76 77 78 78 79 80 80 81 81 82 82 83 83 84 84 84 86 86 86 86 86 87 87 88 88 89 90 91 91 92 92 93 93 93 93 94
95 95 95 96 96 97 97 97 98
Rachit 54

```